

UNIVERSITY AI PROJECT - FULL CONTEXT DOCUMENT

EvoLife

AI-Driven Wildlife & Ecosystem Simulator

A 3D Unity ecosystem in which herbivores and predators survive, compete, reproduce, evolve and respond to environmental pressure. The academic core is the interaction between **multi-agent reinforcement learning**, **genetic evolution**, **ecosystem simulation** and **quantitative evaluation**.

Purpose of this PDF: give any implementation agent enough context to understand the full project before modifying the repository.

Platform: Unity desktop simulation, no XR/VR requirement.

Core AI: Unity ML-Agents PPO + scripted control baseline + genetic evolution.

What EvoLife Is

EvoLife is a real-time wildlife and ecosystem simulator built in Unity. The world contains autonomous herbivores and predators, renewable resources, environmental regions, environmental disturbances and multi-generation reproduction. Individual creatures do not follow a single hard-coded story. Their survival depends on state, sensory information, learned or scripted decisions, inherited genetics and changing ecological conditions.

The project is intentionally designed as an **AI research system** rather than only a visual simulation. The world exists to create measurable decision-making problems: where to move, when to feed, when to drink, when to flee, when to hunt, when to rest, when to reproduce and how these decisions interact with resource pressure and population dynamics.

Academic focus: compare a learned PPO policy with a credible non-learning scripted baseline under the same biological, genetic, environmental and interaction rules. The project must not assume that PPO is superior; results should be measured experimentally.

Primary goals

- Create a living multi-agent ecosystem rather than isolated AI demonstrations.
- Make reinforcement learning central to the project, not a decorative add-on.
- Allow genetics to affect phenotype and produce multi-generation population change.
- Expose environmental scarcity and events that place pressure on populations.
- Record enough analytics to compare policies, survival and genetic change.
- Keep training scenes lightweight and separate from the polished presentation scene.
- Remain understandable and defensible in a university presentation or report.

What the project is not

Not an RPG

There are no quests, inventory systems or player combat loops driving the design.

Not a pure visual sandbox

A beautiful ecosystem without measurable AI behavior is insufficient.

Not one neural network per animal

Policies are shared by species/role; individuals differ through state and genetics.

Not XR/VR

The required presentation is desktop 3D Unity.

Research Question and Experimental Logic

Core research question

How does a PPO-based multi-agent control policy compare with a strong scripted survival policy when both operate under the same sensing limits, action capabilities, biology, genetics, resources and environmental disturbances?

Secondary questions

- Can learned policies improve survival, feeding, drinking, escape or hunting outcomes compared with a deterministic heuristic baseline?
- How do policy choices influence long-term population stability and extinction risk?
- How do inherited genetic traits change over generations under different environmental pressures?
- How robust are different policies to drought, resource scarcity, predator pressure and other disturbances?
- Does adaptation observed over generations reflect genetic selection, learned behavior, or both?

Important methodological rule: do not write code or documentation that assumes PPO wins. The scripted baseline is a legitimate control condition. If the baseline performs better in a scenario, that is still a valid experimental result.

Fair comparison requirements

```
Same world seed
|
+-- Same resource layout
+-- Same biology
+-- Same genetics
+-- Same local sensing limits
+-- Same movement + interaction action space
+-- Same environmental events
+-- Same evaluation metrics
    |
    +-- ScriptedBaseline
    +-- LearnedPpo
```

Paired or matched experiment seeds are preferred so that differences between policies are not dominated by unrelated changes in world initialization.

Expected evidence

The project should ultimately produce survival curves, population time series, generation summaries, trait distributions, event-response metrics and direct policy comparisons. The strongest report will combine quantitative results with interpretable visual examples from the Unity scene.

World, Creatures and Resources

Creature roles

Herbivores

- Search for food and water.
- Flee nearby predators.
- Rest when energy is low.
- Reproduce when eligible.
- Compete indirectly through shared resources.

Predators

- Search for prey and water.
- Pursue and attack herbivores.
- Abandon inefficient hunts.
- Rest when required.
- Reproduce when eligible.

Vital state

Each creature maintains biological state such as health hunger thirst energy age activity alive/dead .

Biology owns the formulas and state transitions. AI observes these values through read-only contracts and requests actions; it must not directly rewrite vital state.

Resources

- **Plants:** consumable food with capacity, depletion and configurable regeneration.
- **Water:** locally detectable drinking sources, normally persistent but configurable for experiments.
- **Biomes:** lightweight logical zones such as grassland, forest, wetland and rocky/dry areas.
- **Day/night:** driven by simulation time rather than real wall-clock time.

Environmental disturbances

Configurable events can include drought, wildfire, heat wave, food boom, disease pressure, predator introduction and predator removal. Events operate through the proper resource, biology and simulation APIs rather than mutating hidden state.

Simulation principle: the environment should be geographically interesting but mechanically readable. Because the agents do not depend on sophisticated NavMesh pathfinding, decorative terrain should avoid mazes, steep cliffs and dense collision obstacles.

Canonical Genome

Unity C# is the canonical runtime genetics implementation. Python genetics code exists as an offline/reference/research implementation of the same conceptual schema. The runtime genome contains nine named traits.

Trait	Meaning in the simulation	Typical gameplay influence
base_movement_speed	Normal locomotion capability	Walking/ordinary movement speed
sprint_speed	Maximum high-effort locomotion	Escape/pursuit potential
vision_range	Local sensory range	How far food, water and creatures can be sensed
maximum_energy	Energy capacity	Available activity/sprint reserve
metabolism_rate	Resource drain tendency	Hunger/thirst/energy pressure
body_size	Body phenotype variable	Visual/optional gameplay adapter, carefully bounded
aggression	Behavioral phenotype variable	Potential future or policy-facing heuristic influence
reproduction_threshold	Readiness requirement	Eligibility for successful reproduction
maximum_age	Lifespan capacity	Age-related mortality limit

Evolution mechanics

- Founders are created from configured trait ranges.
- Successful reproduction combines two parent genomes through canonical crossover.
- Mutation is applied per trait using configurable probability and magnitude.
- All traits are clamped to canonical hard bounds.
- Offspring receive a new ID, parent IDs and generation number.
- There is no artificial global "fitness score" required for evolution.

```
Survival + successful reproduction
|
Parent A ---+--- Parent B
|
crossover
|
mutation
|
bounded genome
|
offspring
|
next generation population
```

Selection emerges from which individuals survive long enough and reproduce successfully under the current ecological conditions.

Multi-Agent Reinforcement Learning + Scripted Baseline

Shared policies

EvoLife should use shared policies rather than one network per creature. A herbivore policy is shared by herbivore agents and a predator policy is shared by predators. Individuals remain different because their observations contain different vital state, local surroundings and genetics.

Control modes

- **ScriptedBaseline:** deterministic/heuristic survival controller used as the experimental control group.
- **LearnedPpo:** Unity ML-Agents PPO policy.
- **Fallback/idle behavior:** safe behavior when the learned agent cannot run, without secretly switching to the scripted policy.

Current observation contract - schema v2

The frozen vector observation contract contains **31 floats**:

Indices	Observation block
0-4	health, hunger, thirst, energy, age
5	own role
6-14	nine canonical normalized genetic traits
15-18	nearest food: local direction X/Z, normalized distance, present flag
19-22	nearest water: local direction X/Z, normalized distance, present flag
23-26	nearest herbivore: local direction X/Z, normalized distance, present flag
27-30	nearest predator: local direction X/Z, normalized distance, present flag

Directions are expressed in creature-local coordinates. Sensing is local, not omniscient. Physics-based nearby-creature sensing requires appropriate colliders.

Current action contract - schema v2

Type	Action	Meaning
Continuous	forward	Move forward/backward in local creature coordinates
Continuous	turn	Yaw left/right
Continuous	sprint_or_effort	Scale between ordinary movement and sprint capability
Discrete branch	none / eat / drink / attack / rest / reproduce_request	Canonical interaction request shared by PPO and baseline

Fairness rule: both PPO and scripted baseline must use the same observation and action capabilities and ultimately execute through the same movement/interaction paths.

Behavior, Rewards and Curriculum

Scripted baseline intent

The baseline should be strong enough to be a meaningful comparison, but should not cheat through global state. It uses the same local sensing contracts as PPO and makes deterministic survival decisions with hysteresis and target persistence.

Herbivore priorities

1. Flee threatening predator.
2. Handle critical survival/rest state.
3. Seek water or food according to need.
4. Rest when energy is low.
5. Wander/explore.

Predator priorities

1. Handle critical survival/rest state.
2. Urgent thirst.
3. Hunt visible herbivore.
4. Ordinary water seeking.
5. Rest or wander.

PPO reward design

A starter reward system may include survival, hunger/thirst relief, health loss, critical need and death signals, but reward values are experimental rather than final. Reward shaping should be tuned through actual training runs and must be monitored for reward hacking.

Do not hard-code reward conclusions. For example, rewarding the "eat" button itself is dangerous; useful rewards should correspond to meaningful successful outcomes when possible.

Training curriculum

```
Stage 1  Movement / orientation
|
Stage 2  Food and water acquisition
|
Stage 3  Predator-prey interaction
|
Stage 4  Resource scarcity
|
Stage 5  Persistent ecosystem
|
Stage 6  Reproduction + environmental events
```

Training should use lightweight scenes/builds, potentially headless or no-graphics, and progressively increase difficulty. The polished demo scene should not be the default training environment.

Multi-Generation Ecosystem

Reproduction turns isolated agents into an evolving population. AI only requests reproduction; the Simulation layer determines whether reproduction actually succeeds.

Eligibility

Typical conditions include:

- Creature is alive and mature.
- Health/energy are sufficient.
- Genetic reproduction threshold is satisfied.
- Cooldown is ready.
- A compatible local mate exists.
- Population cap allows a birth.

Successful birth

```
reproduce_request
|
Simulation validates requester + mate
|
canonical crossover + mutation
|
new CreatureId
new generation = max(parent generations) + 1
ParentA / ParentB stored
|
CreatureSpawner
|
PopulationTracker + lifecycle + analytics
```

Persistent ecosystem vs training support

- **Persistent mode:** death is permanent; reproduction is the source of future generations; extinction may occur.
- **Training-support mode:** controlled founder respawn can be enabled to keep early PPO training from collapsing immediately.

Experiment metadata must clearly distinguish these modes so that training support is not confused with natural ecosystem persistence.

Population pressure

Herbivore and predator population caps/floors, resource abundance, mutation settings and environmental events are experiment-level controls. They should be explicit configuration rather than hidden global constants.

Data Collection Architecture

The Unity simulation should continue even when the analytics backend is unavailable. Analytics observes the simulation; it must not control biology or AI.

Backend

The supporting backend uses FastAPI and SQLite. Unity uses a single backend client and uploads snapshots, lifetime records and generation summaries in batches rather than per frame.

Core metrics

Population

Herbivores, predators, total alive, births, deaths, ratios, extinction.

Lifetime

Age/lifespan, death cause, policy type, generation, parents, genome.

Evolution

Trait mean, variance, min/max by species and generation.

Policy comparison

ScriptedBaseline vs LearnedPpo under matched scenarios/seeds.

Predator/prey

Hunts, attacks, kills, escape success, chase cost when measurable.

Environment

Resource abundance, active events, event recovery metrics when measurable.

Reliability principle

Temporary backend failure should retain queued data for retry within a bounded queue. A long outage may eventually require an overflow policy so the Unity process cannot consume unlimited memory. For final scientific runs, local persistence can be added if stronger durability is required.

Evaluation outputs

- JSON and CSV exports suitable for report graphs.
- Population and trait time series.
- Paired-seed baseline/PPO comparison tables.
- Event-response summaries such as minimum population and recovery time.
- Model/checkpoint identifiers recorded for PPO experiments.

Reproducible Runs

Experiments should be represented as serializable configuration rather than ad-hoc Inspector changes. Important fields include:

Population

Initial herbivores/predators, caps, floors, persistent vs training mode.

Resources

Resource abundance and plant regeneration multiplier.

Genetics

Mutation probability and magnitude, deterministic seed streams.

Environment

Day length, enabled/scheduled ecological events.

AI

Herbivore policy, predator policy, model ID/checkpoint.

Run control

Scenario ID, random seed, duration and stopping conditions.

Starter scenario ideas

normal_control reduced_food drought fast_predators high_mutation low_mutation predator_pressure recovery_after_event

Reproducibility: derive separate deterministic seed streams for founder genomes, reproduction/mutation, resource placement, event schedules and scripted wandering. Unity physics may still have some practical nondeterminism; document this rather than pretending perfect determinism.

Experiment lifecycle

```
pause simulation
|
validate/load configuration
|
configure environment + lifecycle + deterministic seeds
|
place resources
|
spawn founders
|
start analytics run
|
mark Running
|
unpause simulation
|
stop by duration / extinction / manual request
|
finish analytics + pause
```

Desktop Demo Experience

Presentation world

The final demo world should look like a stylized ecological research diorama: open central grassland with irregular forest, wetland and dry/rocky regions around it. The world should be attractive but remain mechanically open enough for direct AI locomotion.

Visual priorities

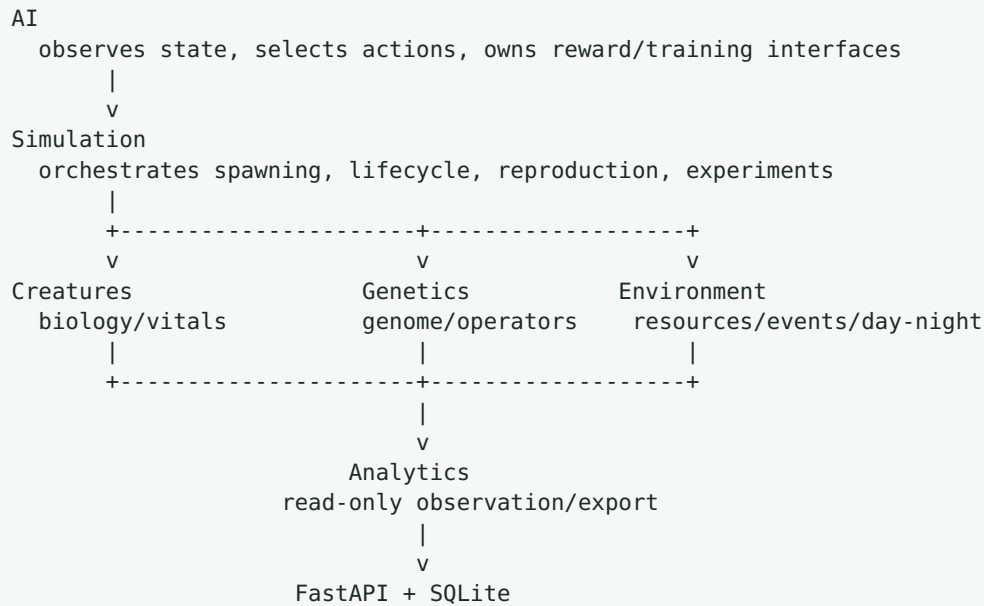
- Creatures must be readable and predators/herbivores distinguishable by silhouette, not only color.
- Food resources and water should be visually identifiable from an overview camera.
- Biomes should not look like perfect circles or game-board quadrants.
- Decorative vegetation should usually have no collider.
- Day/night and event visuals must not invent mechanics that do not exist.

Desktop UI/debug tools

- Free-fly/orbit camera and creature selection.
- Creature inspector with identity, vitals, genetics, policy and AI action state.
- Toggleable local sensing/debug visualization.
- Pause/resume and simulation speed controls.
- Manual environmental event triggers.
- Live population/resource/evolution dashboard and lightweight charts.

Presentation rule: when choosing between "more beautiful" and "easier to understand the AI behavior," choose easier to understand the AI behavior.

Module Boundaries



Ownership rules

- **Creatures:** owns vital state and biological formulas.
- **Genetics:** owns canonical genome schema, crossover, mutation and phenotype decoding.
- **AI:** owns observations, actions, policies and training reward logic.
- **Simulation:** owns spawning, lifecycle, population orchestration, reproduction and experiment sequencing.
- **Environment:** owns plants, water, biomes, day/night and ecological event effects on environment state.
- **Analytics:** observes through read-only contracts; it does not control gameplay.
- **UI/Presentation:** reads state and issues supported requests; it does not become a hidden simulation owner.

Suggested repository structure

```

Assets/EvoLife/
  Scripts/
    AI/
    Creatures/
    Genetics/
    Environment/
    Simulation/
    Analytics/
    UI/
    Common/
  Prefabs/
  ScriptableObjects/
  Materials/
  Scenes/
  Tests/

Backend/
Training/
Docs/

```

How Future Implementation Agents Should Work

Before changing code, an agent should inspect the actual repository and read the architecture, boundaries and domain-specific docs. This PDF explains project intent, but the repository is the source of truth for the latest implementation details.

Non-negotiable constraints

- Do not create a second AI framework, second genetics schema, second analytics backend or duplicate simulation manager.
- Do not modify biology state directly from AI, UI, analytics or environment code.
- Do not give one policy omniscient/global information that the comparison policy cannot access.
- Do not silently change PPO observation/action dimensions after meaningful training has begun.
- Do not commit trained models as proof of quality without real evaluation.
- Do not claim Unity tests passed if the Unity Editor was unavailable.
- Do not make the polished demo scene the default heavy training environment.
- Do not add XR/VR requirements.

Testing expectations

- Pure logic should be covered by EditMode/unit tests where possible.
- Unity-only wiring and physics behavior need Editor/PlayMode verification.
- Python genetics and experiment utilities should run under pytest.
- FastAPI backend changes should run backend pytest.
- Cloud agents must clearly distinguish tests written from tests actually executed.

What should remain the student's own AI work

The final PPO training process, reward experimentation, training-curve interpretation, reward-hacking diagnosis, curriculum decisions, checkpoint selection and final scientific interpretation should remain understandable and defensible by the student. Agents can build infrastructure, tests and starter configurations, but the central AI experimentation should not become a black box.

Definition of success: EvoLife is successful when it can run a reproducible ecosystem experiment, compare learned and scripted behavior fairly, show multi-generation genetic change, survive environmental perturbations, export evidence and make the AI behavior visually understandable in Unity.

Compact Context for an Agent

If an agent only reads one page, it should retain the following:

1. **EvoLife is a university AI project** centered on PPO multi-agent reinforcement learning, genetic evolution and ecosystem simulation.
2. **There are herbivores and predators** with vital state, local sensing, movement, interactions, death and reproduction.
3. **Policies are shared by role/species**; individuals differ through state and a nine-trait genome.
4. **PPO and the scripted baseline must be comparable fairly** under the same sensors/actions and world rules.
5. **The canonical PPO contract is observation schema v2 (31 floats)** and action schema v2 (forward, turn, effort + six-way interaction branch).
6. **Simulation owns reproduction/lifecycle**; genetics owns crossover/mutation; biology owns vitals; environment owns resources/events; analytics is read-only.
7. **Experiments must be reproducible** with explicit configurations, seeds, scenario/model IDs and analytics.
8. **Training should be progressive and lightweight**; the polished scene is for demonstration/evaluation, not default training.
9. **Do not assume PPO wins.** Measure it.
10. **Do not invent parallel architecture.** Inspect and extend the current repository.

Recommended first action for any new coding agent: read `Docs/ARCHITECTURE.md`, `Docs/AGENT_BOUNDARIES.md`, the relevant domain docs, and inspect the latest implementation before editing.

This document is a consolidated reconstruction of the EvoLife project description and the architectural/research context established during development. It is intended as a handoff/context artifact, not as a substitute for the repository's latest code and documentation.